ACRME Architecture Decision Records (ADRs)

> ## ⚠️ ARCHIVED — SUPERSEDED by the split, v2.2 ADRs (2 Sep 2026)
>
> This **consolidated** ADR document is **archived**. The authoritative decision records are the individual, v2.2-aligned ADRs in `docs/adr/` : **ADR-001** (region selection), **ADR-002** (quota & capacity — **single governed pool**), **ADR-003** (capacity during DR), **ADR-004** (forecast & increase), and **ADR-005** (distributed DR reference model). See `docs/adr/README.md` for the index. This consolidated file has had its region references corrected (Belgium → **Switzerland North**) but does **not** carry the full single-pool (QUA-004) and max-not-sum (DR-017) reconciliation — use the split ADRs and the FDD/TDD for current decisions.

Project: Azure Capacity Reservation Management Engine (ACRME)
Classification: Principal Cloud Architect — Architecture Governance
Version: 1.2
Date: August 2026
Status: Accepted
Change note (v1.2): Made this a self-contained, reference-free document — removed external document and section citations so it can be read without any companion document — and added rendered architecture diagrams (placement pipeline, input modes, quota-group model, engine state machine, emergency-transfer tiers, capacity-increase lifecycle). **(v1.1):** Added the region-classification model, Scenario 1/2 input modes, the exception workflow, the validation-rule framework, capacity-hold concurrency and corrected scoring (ADR-001); group accounting formulas, the dual-validation detector and `groupType` preview status (ADR-002); the five-state engine state machine, the `EngineModeState` entity and DR-activation semantics (ADR-003); and the 10-step steady-state lifecycle with the auto-decrease exclusion (ADR-004).

About This Document

An **Architecture Decision Record (ADR)** captures a single significant architectural decision, the context that forced the decision, the options considered, the choice made, and its consequences. ADRs are immutable once accepted — a superseding decision is recorded as a new ADR rather than editing the original.

This document records the four foundational ACRME architecture decisions:

ADR	Title	Status	Supersedes
ADR-001	Region Selection	Accepted	—
ADR-002	Quota and Capacity Management	Accepted	—
ADR-003	Capacity Management during Disaster Recovery (DR)	Accepted	—
ADR-004	Forecast and Increase of Capacity and Quota	Accepted	—

Evidence tags used throughout: `[Documented]` (traceable to Azure platform behaviour or documentation), `[Decided]` (an explicit ACRME design choice recorded in this ADR set), `[Derived]` (a logical consequence of a documented constraint or decision), `[Assumed]` (architectural judgement pending proof-of-concept validation).

ADR-001 — Region Selection

Status: Accepted
Date: August 2026
Deciders: Principal Cloud Architect, Platform Engineering, DR Owner
Related constraints: HC-1..HC-10 (hard constraints); VR-1..VR-11 (validation rules)

Context

Customers request capacity by supplying either a specific Azure region or an Azure geography (e.g. “US”, “Europe”, “Middle East”). The engine must derive three environment regions — **Prod**, **NonProd/CVAL**, and **DR** — that satisfy isolation, resilience, capacity, quota, and data-residency requirements.

Forces at play:

- Azure regions differ in SKU availability, zone count, capacity headroom, and quota posture. A naive “pick the nearest region” approach silently produces single-zone or capacity-starved placements.
- Some regions are **Restricted Capacity Regions** (sovereign clouds, limited-SKU regions) that must never be selected by automated placement.
- Certain geographies (notably the **Middle East**, with only UAE North and Saudi Arabia Central) do not have enough Standard regions to satisfy a three-region model in-geo, forcing a governed cross-geo DR path.
- Placement decisions must be **deterministic, auditable, and replayable** — the same inputs must always produce the same output, and every decision must be reconstructable for compliance.

Decision

Adopt a **staged, constraint-then-score placement pipeline** driven by environment-type-specific scoring formulas:

1. **Stage 1 — Eligibility pre-filtering.** Reduce all Azure regions to Standard Capacity Regions within the customer's geography.
   - **HC-9 STANDARD_REGION_ONLY** excludes Restricted regions before scoring. `[Decided]`
   - **HC-8 GEOGRAPHY_CONTAINMENT** confines the Prod anchor to the customer's chosen geography. `[Derived]`

2. **Stage 2 — Hard-constraint gate.** Each surviving region must pass **HC-1..HC-7 and HC-10** (region separation, capacity floor, quota floor, DR separation class, zone availability, DR coverage floor, DR floor integrity, cross-geo extension approval). Any failure excludes the region from scoring entirely — hard constraints are binary gates, never scored penalties. `[Decided]`

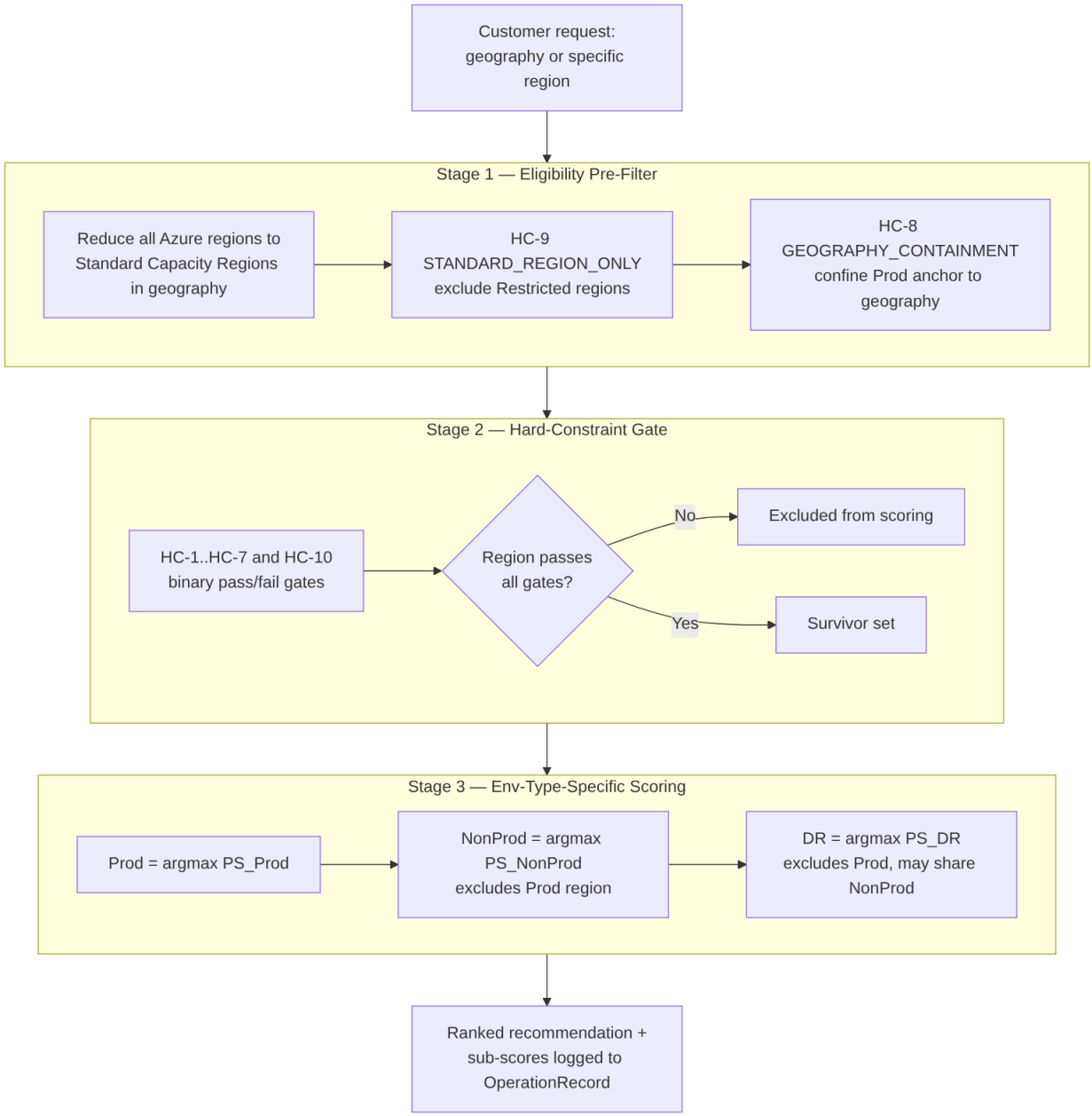
3. **Stage 3 — Environment-type-specific scoring.** Rank survivors using three formulas sharing default weights (α=0.30, β=0.20, γ=0.25, δ=0.15, ε=0.10) but with env-type-specific semantics for α (capacity signal) and δ (DR readiness): `[Decided]`
   - **Prod:** `Prod_region = argmax(PS_Prod(r))` over eligible Standard regions in-geo.
   - **NonProd/CVAL:** `argmax(PS_NonProd(r))` over survivors, excluding the Prod region.
   - **DR:** `argmax(PS_DR(r))` over survivors, excluding the Prod region (but **may** share with NonProd).

4. **Selection order is sequential — Prod → NonProd → DR** — not joint optimization. With 3–4 regions the greedy sequential result equals the joint-optimal result in nearly all practical cases, while remaining transparent and auditable. `[Decided]`

5. **Middle East special handling.** Prod and NonProd are chosen in-geo via `argmax(PS_Prod)` over {UAE North, Saudi Arabia Central}; DR is assigned to **Switzerland North** via the only approved Cross-Geo Extension paths (Saudi Arabia → Switzerland North, UAE North → Switzerland North). Switzerland North must itself pass HC-1..HC-10; if it fails, the placement is rejected with an ops alert — the engine never silently substitutes another region. `[Decided]`

6. **Determinism & audit.** All candidate sets, sub-scores, the winning score, and the active `PlacementPolicy` version are written to the `OperationRecord` for replay (VR-8, VR-9). Even the 3-region edge case (single eligible candidate) runs the full scoring path so the score is logged as a capacity-health signal. `[Decided]`

7. **Operating mode.** In Phase 1 the engine runs region selection in **recommendation/shadow mode** — it produces and logs the ranked recommendation but does not autonomously place. Autonomous placement is gated on POC validation of the scoring formulas.



Region Classification Model (normative)

Every Azure region carries exactly one classification tier, stored in `PlacementPolicy` as config-as-code (versioned, auditable, replayable) — classification is a **governance decision, not a live capability query**. `[Documented]`

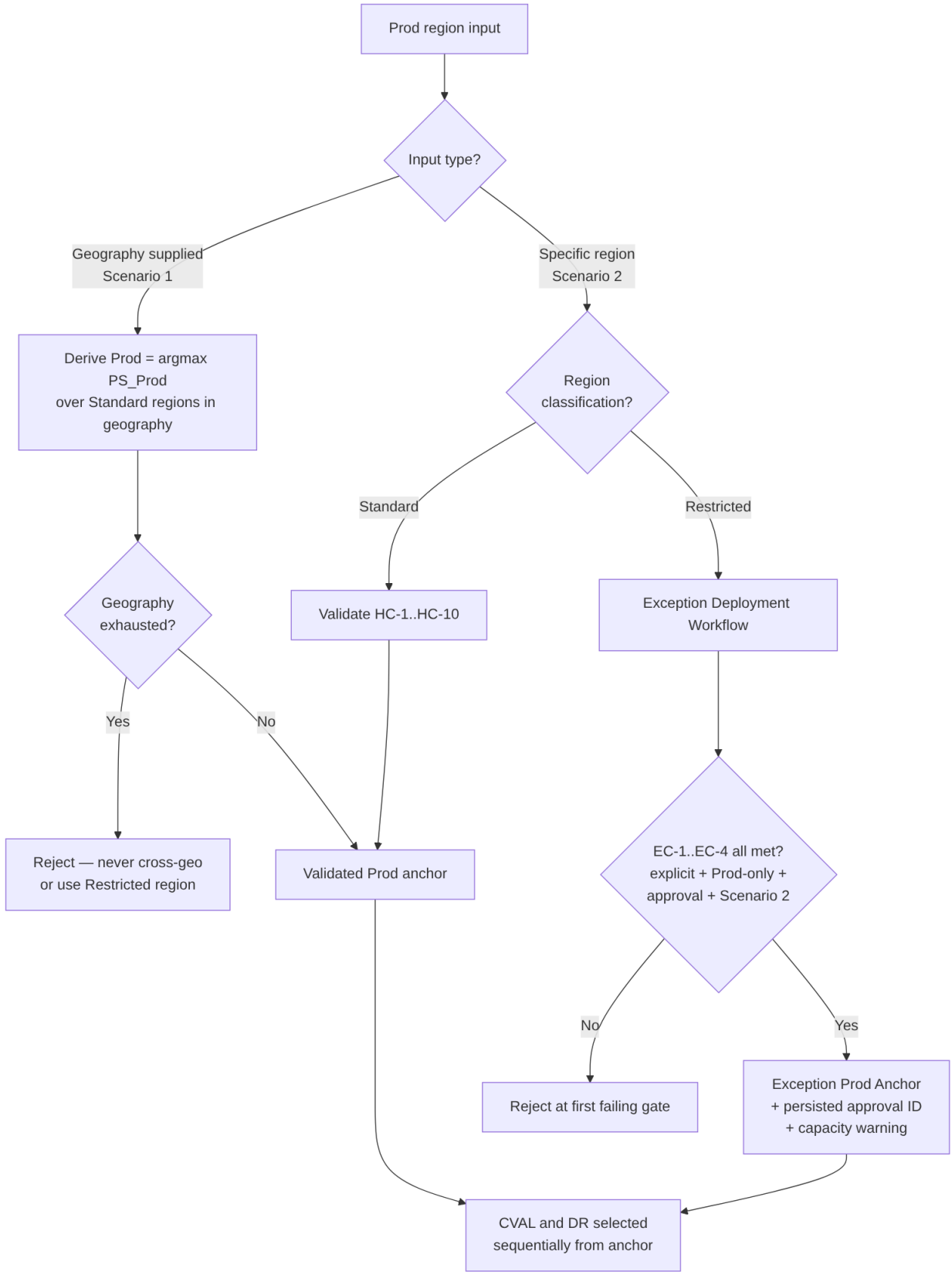
Tier	Eligibility	Regions
Standard Capacity Region	Eligible for dynamic selection, scoring, and all env assignments (Prod/CVAL/DR) — the only regions that enter the pipeline	NA: West US 3, Central US, Canada Central · EU: Sweden Central, Switzerland North · ME: Saudi Arabia, UAE North · APAC: Japan East, Southeast Asia, Australia East
Restricted Capacity Region	Exception-only (Scenario 2 + Prod + approval); never scored, ranked, or recommended	East US 2, North Europe, West Europe, East Asia, Australia Southeast (all: Azure physical capacity constraint)
Cross-Geo Extension Region	DR-only, for geographies that cannot meet the 3-region minimum in-geo	Saudi Arabia → Switzerland North; UAE North → Switzerland North (Middle East only)

Restricted regions are excluded by a **pre-filter ahead of all hard constraints** (HC-9), never as a scoring penalty.

Prod Region Input Modes

The Prod region is the anchor; CVAL and DR are selected sequentially from it. The customer supplies the anchor one of two ways, and both converge on a single validated Prod anchor: `[Documented]`

- **Scenario 1 — geography supplied.** The engine derives Prod via `argmax(PS_Prod)` over Standard Capacity Regions **in that geography only**. Ties break deterministically by the Standard-region list order for the geography; the first-listed region is the deterministic cold-start default when no snapshot exists. Geography exhaustion → reject (never silently cross-geo or use a Restricted region).
- **Scenario 2 — specific region supplied.** If Standard, validate against HC-1..HC-10 and adopt as the anchor (`PS_Prod` used only for post-selection validation). If Restricted, route to the Exception Deployment Workflow.



Exception Deployment Workflow (Scenario 2 — Restricted region)

A Restricted region is used only if **all four** conditions hold; failure rejects at the first failing gate: `[Doc-umented]`

#	Condition	Check
EC-1	Explicit request	Customer named the region; engine never recommended it
EC-2	Production only	CVAL and DR may **never** use a Restricted region
EC-3	Exception approval	A named, revocable approval record exists for the custom-er–region pair
EC-4	Scenario 2 input	Restricted regions can never be engine-derived

On success the region becomes the **Exception Prod Anchor**, the placement is marked an **Exception Deployment**, a capacity-constraint warning is emitted to the caller, and the approval ID + restriction status are mandatory `OperationRecord` fields (commit blocked if absent). CVAL/DR still select from Standard regions via normal scoring.

Validation Rule Framework (VR-1..VR-11)

Rule	Check	Failure action
VR-1	Region exists in classification list	Reject if unknown
VR-2	Automated placement uses Standard only	Exclude before scoring if Restricted
VR-3	Scenario 2 Restricted: EC-1..EC-4 all met	Reject at first failing condition
VR-4	Scenario 1 derived Prod in-geo Standard	Geography-scoped exhaustion error
VR-5	Standard region passes HC-1..HC-10	Exclude; exhaustion error if all excluded
VR-6	Middle East DR = Switzerland North via approved path	Block with ops alert if Switzerland North fails HC-1..HC-10
VR-7	Exception approval ID persisted before commit	Block commit if absent
VR-8	Capacity-constraint warning emitted	Block commit if suppressed
VR-9	Snapshot age within policy limit	Trigger targeted ARM refresh
VR-10	Capacity hold acquired before commit	Block commit if hold absent
VR-11	Restricted regions absent from all recommendation outputs	Post-scoring filter (defence-in-depth)

Capacity Holds & Concurrency

Before returning a committed assignment the engine creates a **capacity hold** keyed by region, SKU, zone, environment, and policy version, using **optimistic concurrency**; the hold expires if Azure provisioning does not begin. This closes the concurrent-placement race. `[Documented]`

Corrected Scoring Model (pilot)

- Default weights retained for pilot comparison: `α=0.30, β=0.20, γ=0.25, δ=0.15, ε=0.10`; every component is clamped `Clamp(x) = max(0, min(1, x))`. `[Assumed]`
- To avoid double-counting the same signal under α and δ, the CVAL/NonProd formula is proposed as `PS_NonProd = 0.35·Capacity + 0.25·Quota + 0.25·Distribution + 0.05·DR_Overflow_Integrity + 0.10·Zones`. `[Assumed]`

- Distribution uses **demand units, not customer count**: `Distribution = 1 − Region_Assigned_Demand / Total_Assigned_Demand`. `[Assumed]`
- Revised weights are proposed, not empirically validated — advisory until pilot measurement.

Governance & Compliance Controls

- The classification list lives in `PlacementPolicy`; any change requires a policy-version increment, a Decision Log entry, and **replay of the prior 30 days of placements** against the new classification before activation. `[Documented]`
- Exception approval records are revocable engine artefacts; a revoked approval blocks future exception deployments for the customer–region pair with no code change.
- Every classification change is audited with approver identity, timestamp, previous/new classification, and affected geography.
- Switzerland North's regional capacity-planning targets must include potential Middle East DR demand on top of in-geo Europe demand.

Consequences

Positive:
- Deterministic, replayable placement with a complete audit trail satisfies compliance and post-incident review.
- Hard constraints guarantee no placement ever violates isolation, zone, quota, or residency rules regardless of score.
- Restricted regions are structurally impossible to select automatically; exceptions flow through a governed Scenario 2 workflow with a persisted, revocable approval ID.
- Env-type-specific formulas let Prod optimise for capacity/isolation while DR optimises for overflow readiness.

Negative / trade-offs:
- Three formulas plus 16 per-CRG-type `RegionalSnapshot` fields increase implementation and snapshot-maintenance cost.
- Sequential selection is greedy; if the region count ever exceeds 6, joint optimization should be revisited.
- Cross-geo extension paths are a manual governance artefact — adding a path requires a PlacementPolicy update, governance approval, and a Decision Log entry.
- Worked scoring examples remain a known gap until per-CRG-type inputs are finalised.

Neutral:
- CRG_Score (Regional Capacity Weight) is demoted from a primary scoring input to a monitoring-only signal.

Alternatives Considered

Alternative	Why Rejected
Joint optimization (select all three regions simultaneously)	Higher combinatorial complexity; opaque and harder to audit; identical result to sequential for 3–4 regions. `[Decided]`
Single generic PS(r, env_type) formula	α and δ have fundamentally different meanings per env type; a single formula needs so many branches it becomes three formulas anyway. `[Decided]`
Geographic distance as a scored soft objective	Operators already choose geographically independent region sets at design time; HC-4 as a hard constraint is sufficient and simpler. `[Decided]`
Silent fallback region for failed Middle East DR	Violates governance and residency guarantees; the engine must fail loudly and require an approved extension path. `[Documented]`

ADR-002 — Quota and Capacity Management

Status: Accepted
Date: August 2026
Deciders: Principal Cloud Architect, Platform Engineering, FinOps
Related constraints: HC-2, HC-3, HC-7 (hard constraints)

Context

Each customer needs three CRGs per region (Prod, NonProd, DR). Azure tracks compute quota per subscription per region per SKU family; quota exhaustion silently blocks CR creation and VM deployment. The engine must:

- Prevent a NonProd surge from consuming quota that Prod CR creation needs (isolation).
- Make emergency capacity transfer (see ADR-003) **quota-neutral** where possible — i.e. reshuffle capacity without waiting hours for an Azure quota-increase approval during a crisis.
- Provide clean per-environment cost attribution for chargeback.
- Enforce a protected DR reservation that NonProd growth can never erode.

Azure **Quota Groups** (Preview) allow multiple subscriptions/CRGs to share a group-level quota pool, but Azure does **not** natively support intra-group sub-reservations — so any sub-partition of a group pool must be enforced by the engine.

Decision

Adopt a **Two-Quota-Group-per-region model with an engine-enforced DR floor**:

1. **Two groups per region:** one **Prod-only** group and one **shared NonProd+DR** group.
   `[Decided]`
   `Prod_Group_Limit(R)  = base_subscription_quota_limit × (1 + emergency_transfer_headroom_vcpu / base_limit)`
   `NonProd_DR_Group_Limit(R)  = base_subscription_quota_limit × (1 + emergency_transfer_headroom_vcpu / base_limit)`

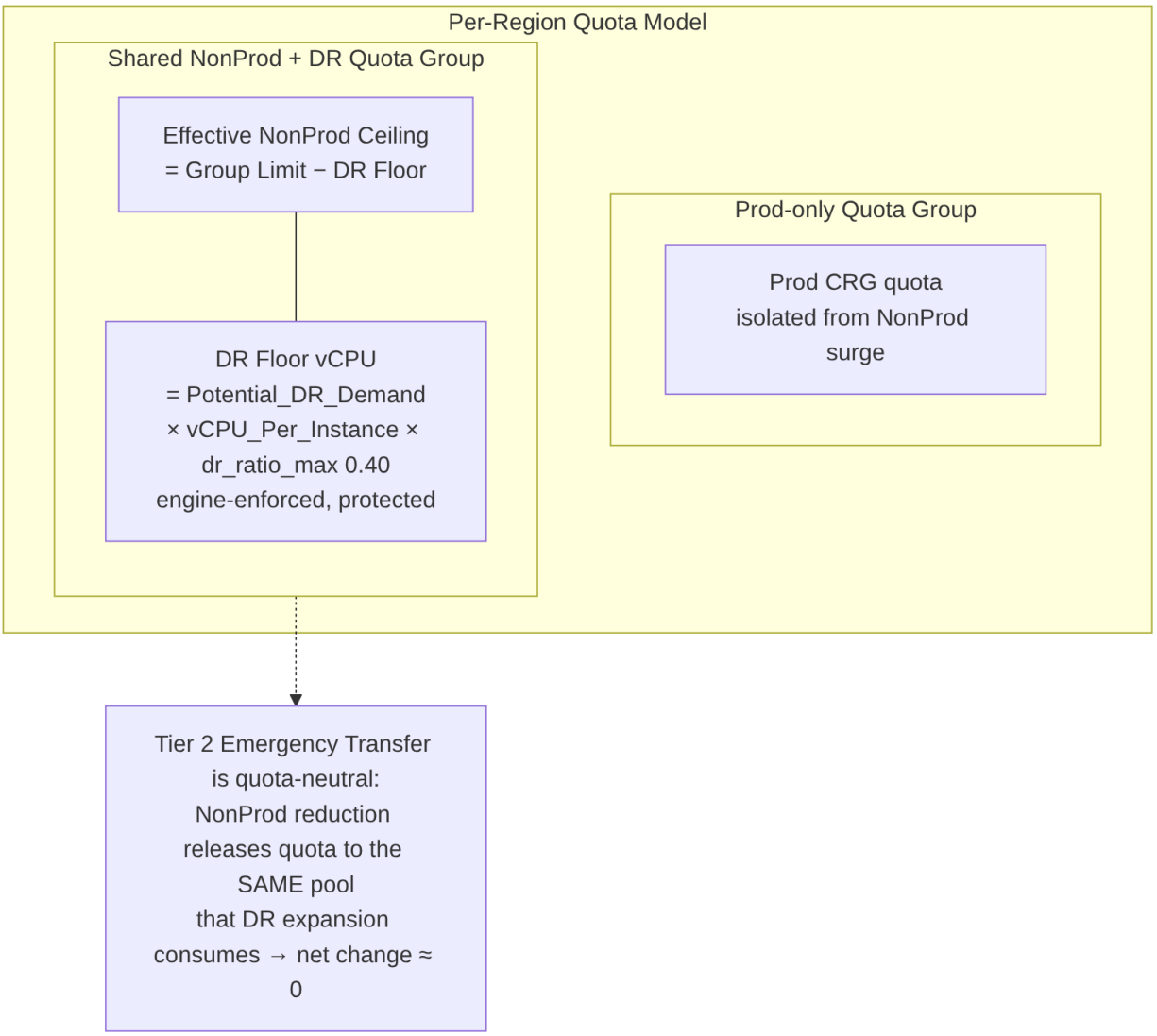
2. **Engine-enforced DR floor** inside the NonProd+DR group (Azure has no native sub-reservation):
   `[Decided]`
   `DR_Floor_vCPU(R)  = potential_dr_demand(R) × vCPU_per_instance × dr_ratio_max`
   `Effective_NonProd_Ceiling(R)  = NonProd_DR_Group_Limit(R) − DR_Floor_vCPU(R)`
   The floor is always sized at **`dr_ratio_max` (0.40)** — the upper bound of the DR ratio range — so it never needs to grow if the operating ratio is later raised (0.30 → 0.40). Only actual Prod demand growth expands the floor. `[Decided]`

3. **Hard-constraint enforcement at placement:**
   - **HC-2 CAPACITY_FLOOR** — CR headroom ≥ `2 × requested_vm_count × SKU_vCPU`.
   - **HC-3 QUOTA_FLOOR** — env-type-aware group headroom check (Prod group / effective NonProd ceiling / DR headroom). Reads from the `QuotaGroup` entity, not legacy per-subscription `QuotaRecord`. `[Decided]`
   - **HC-7 DR_FLOOR_INTEGRITY** — rejects NonProd placement that would push `nonprod_quota_used` above the effective ceiling, evaluated **after** HC-3. `[Decided]`

4. **Legacy per-subscription quota tracking is retained for monitoring/audit only** and superseded for all placement and scoring checks by group-level fields. `[Decided]`

5. **Continuous protection:** the reconciliation loop watches for floor violations and emits `DRFloorViolationDetected`; NonProd CRG scale operations that approach the ceiling are operator-gated.

Worked topology example: Prod group 128 vCPU; NonProd+DR group 80 vCPU; DR floor 32 vCPU → effective NonProd ceiling 48 vCPU.



Group Accounting Formulas (normative)

All four are **engine accounting controls** — they do not create a native Azure sub-reservation: `[Documented]`

```
DR_Floor_vCPU             = Potential_DR_Demand × vCPU_Per_Instance × DR_Ratio_Max
Effective_NonProd_Ceiling = NonProd_DR_Group_Limit − DR_Floor_vCPU
NonProd_Headroom          = Effective_NonProd_Ceiling − NonProd_Used_vCPU
Group_Headroom            = Group_Limit − Group_Used
```

Exact Enforcement Controls

- Every **NonProd** increase performs a group check **and** a subscription check. `[Documented]`
- Every **DR** increase performs a group check, subscription check, SKU check, capacity check, **and** active-incident check.
- A **separate detector** recalculates the DR floor from authoritative assignment/allocation data. Any disagreement between the command-time and detector calculations **disables automatic Non-Prod expansion** (fail-safe) and raises `DRFloorViolationDetected`.
- Group propagation is **polled** — no fixed propagation SLA is assumed.

`groupType` Preview Dependency

The Quota-Group `groupType` property (`AllocationGroup` = advisory vs `EnforcedGroup` = enforced) is **preview-only** in the Azure Quota REST API and is not GA. `[Documented]` The engine's accounting controls are deliberately **engine-level and do not depend on `groupType` enforcement**. If `EnforcedGroup` is ever relied upon to drop the engine's own subscription-level check, that dependency must pass a preview-acceptance proof-of-concept and a recorded decision first. Engineering constraint: pin the exact preview API version in all quota-group calls and add version-drift detection to the platform health check.

Consequences

Positive:
- Prod isolation is enforced at the Azure control-plane level — a NonProd surge cannot starve Prod.
- Tier 2 Emergency Capacity Transfer becomes **truly quota-neutral** (ARM operations only, no Azure approval gate) because NonProd and DR draw from the same group pool (see ADR-003).
- Clean chargeback: one Azure billing record per group per region.
- The DR floor guarantees a protected reservation NonProd can never erode.

Negative / trade-offs:
- The DR floor is a **soft ceiling the engine must maintain continuously** — Azure won't enforce it. A bug could allow NonProd to breach the floor; mitigated by `DRFloorViolationDetected` alerting and operator gates. `[Decided]`
- Sizing the floor at `dr_ratio_max` over-reserves quota slightly at lower operating ratios — a deliberate reliability-over-cost trade, reviewed quarterly. `[Decided]`
- `Quota_Score` and HC-3 must be environment-type-aware (three code paths).
- **Hard dependency on Azure Quota Groups GA** — validated by proof-of-concept before rollout; if `groupQuotas` returns 404 in the target tenant/region, escalate to Azure Support.

Alternatives Considered

Alternative	Why Rejected
Single shared quota pool (all three CRGs)	Breaks Prod isolation — a NonProd surge can consume Prod's quota. `[Decided]`
Three separate groups (one per CRG)	Maximum isolation but makes Tier 3 transfer non-atomic — releasing NonProd quota lands in the wrong group, forcing an Azure quota request mid-crisis (hours of RTO impact). `[Decided]`
Per-subscription quota only (legacy)	Cannot make emergency transfer quota-neutral without risky cross-subscription coordination. `[Decided]`
DR floor sized at current ratio	Undersizes the floor if the ratio is later increased, blocking DR expansion. `[Decided]`

ADR-003 — Capacity Management during Disaster Recovery (DR)

Status: Accepted
Date: August 2026
Deciders: Principal Cloud Architect, DR Owner, Platform Engineering, Security
Related constraints: HC-1, HC-4, HC-6 (hard constraints)

Context

When a primary region fails, customers must recover in their DR region within a committed RTO. Pre-positioning a full 1:1 DR reserve is prohibitively expensive, yet under-provisioning risks a failed fail-over. During a crisis, waiting on Azure quota-increase approvals (hours) is unacceptable. The engine must also strictly separate **routine growth** from **destructive crisis operations** so that reconciliation can never accidentally trigger VM disruption.

Two capabilities are needed:
- A way to reuse otherwise-idle NonProd capacity as DR overflow.
- A safe, tiered escalation model for expanding DR capacity during a declared event.

Decision

Adopt **NonProd/DR co-location with a coverage floor**, a **formal two-mode engine state**, and a **three-tier emergency transfer model**:

1. **NonProd and DR may share a region (HC-1 constraint removed).** This lets the NonProd CRG's `effective_free` (after `dr_overflow_reserve`) count as DR overflow headroom. `[Decided]`
   - **HC-6 DR_COVERAGE_FLOOR** guarantees the combined pool can absorb demand before DR placement is accepted:
   `dr_crg_free_slots(R) + nonprod_crg_effective_free(R) ≥ prod_vm_count × dr_ratio_max`
   - **HC-4 DR_SEPARATION_CLASS** still guarantees Prod and DR sit in non-correlated failure domains (HIGH separation for non-paired regions).

2. **Two separate operating systems gated by `engine_mode`:** `[Decided]`
   - `STEADY_STATE` — organic growth via the reconciliation loop and `CapacityIncreaseRequest` (see ADR-004). Emergency Transfer is **rejected** in this mode.
   - `DR_EVENT_ACTIVE` — crisis operations only. The auto-increase trigger is **suppressed** in this mode.
   Mode transitions are operator-gated with dual approval (state machine `EngineModeState`), never automatic. `[Decided]`

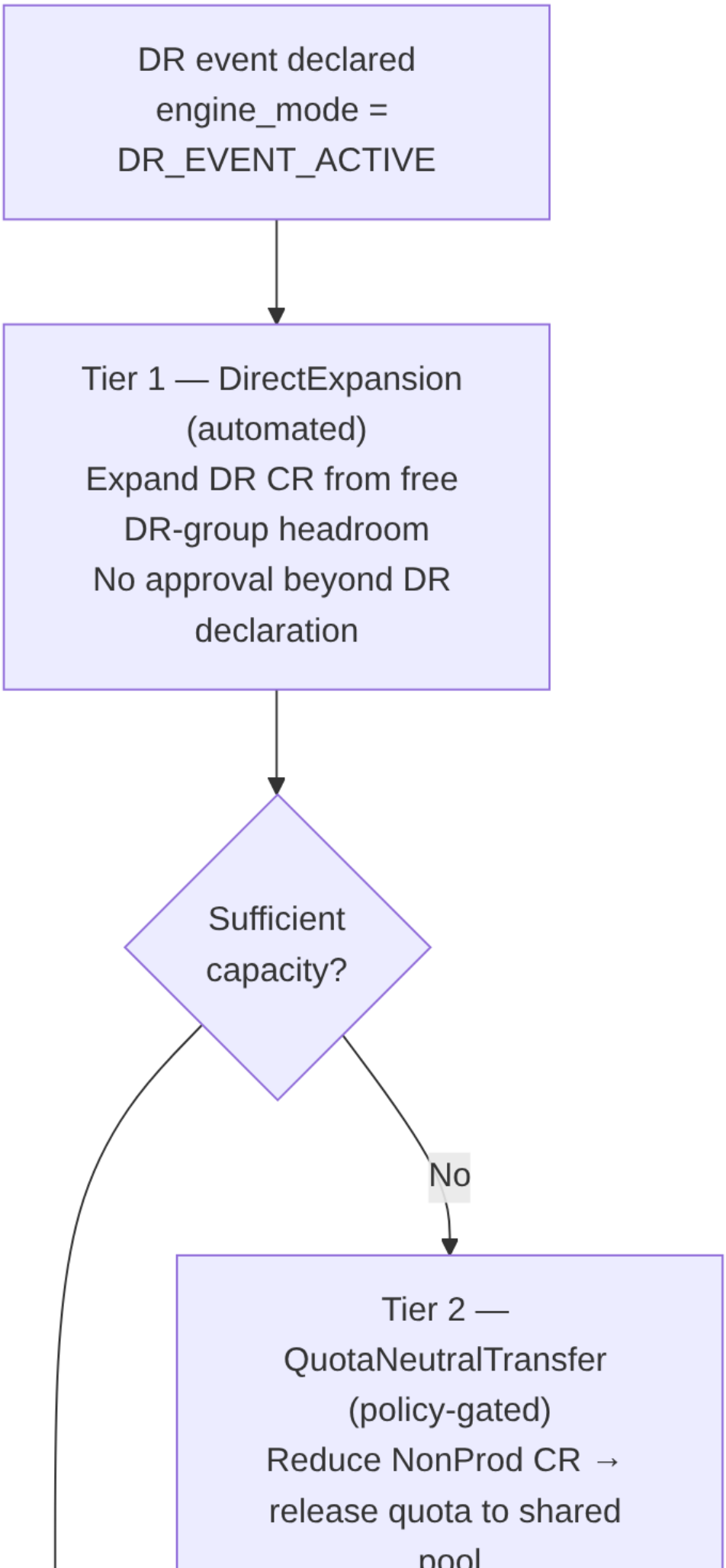
3. **Three-tier Emergency Capacity Transfer escalation:** `[Decided]`
   - **Tier 1 — DirectExpansion (automated):** expand DR CR quantity using free headroom in the DR group. No approval beyond DR-event declaration.
   - **Tier 2 — QuotaNeutralTransfer (policy-gated):** reduce a NonProd CR (releasing quota to the shared NonProd+DR group pool) and expand the DR CR from that same pool. Net group headroom change ≈ 0 — **quota-neutral, no VM execution-state change** (only NonProd SLA is removed). This is possible only because of the two-group model (see ADR-002).
   - **Tier 3 — DestructiveTransfer (dual approval + elevated RBAC):** the only tier that modifies VM-to-CRG associations. The operator supplies an explicit `vm_disassociation_list` (no automated VM selection in Phase 1); executed via 6-step Path B with Path A fallback per VM. VMSS entries are rejected in Phase 1.

4. **Quota-neutral math (Tier 2):**

```
NonProd CR reduction  → releases quota to shared NonProd+DR GROUP pool
  DR CR expansion       → consumes from the SAME pool
  ⇒ net group headroom change ≈ 0  (ARM operations only; no Azure quota approval)
```

5. **DR reserve sizing** is `30–40%` of Prod (`dr_ratio_min=0.30` , `dr_ratio_max=0.40` , `dr_ratio_target=0.35`); the protected floor uses `dr_ratio_max` (see ADR-002).

6. **Phase-1 safety posture:** Tier 2 is approval-gated; **Tier 3 is blocked** pending the consumer-credential model and the engine-mode state machine. No invented SLA — propagation/approval times are measured and reported as unknown until observed.

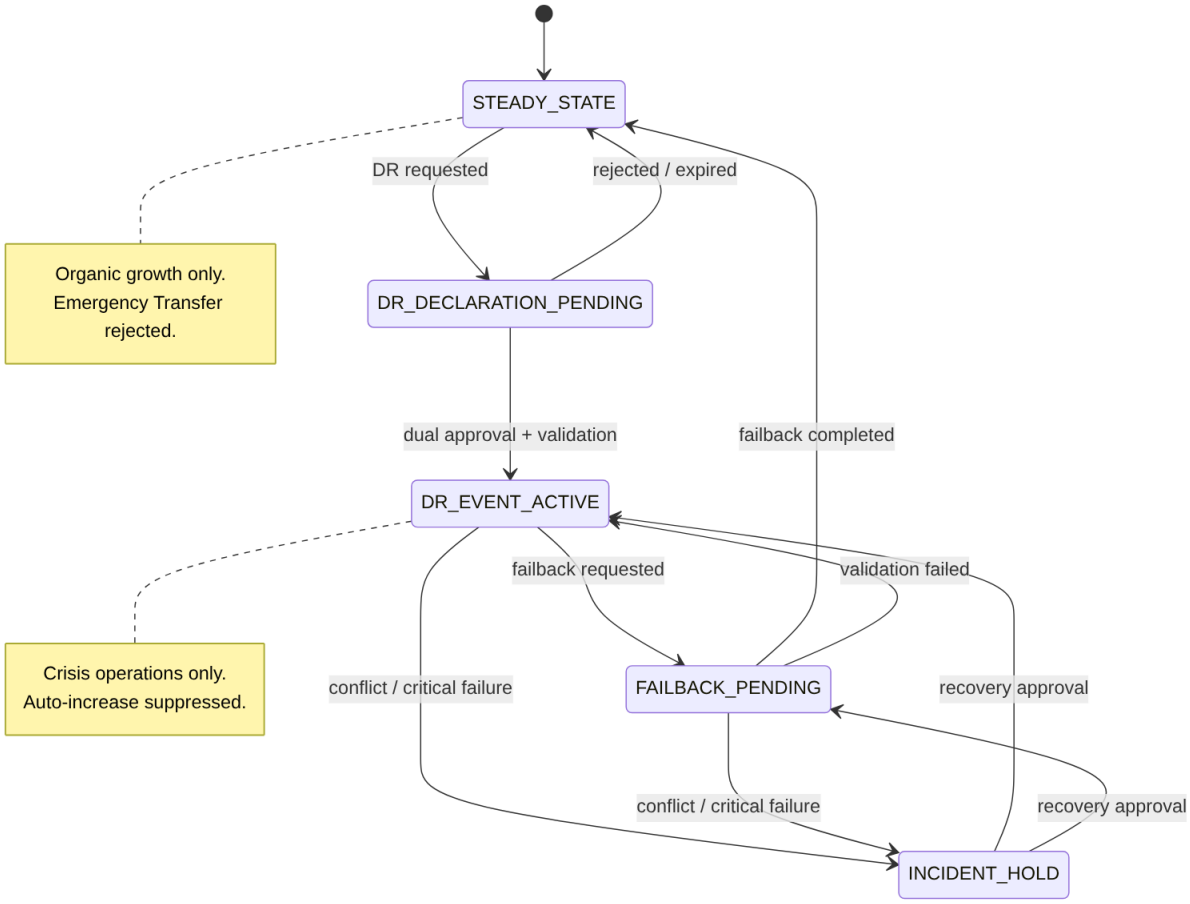

DR event declared
engine_mode =
DR_EVENT_ACTIVE
Tier 1 — DirectExpansion
(automated)
Expand DR CR from free
DR-group headroom
No approval beyond DR
declaration
Sufficient
capacity?
No
Tier 2 —
QuotaNeutralTransfer
(policy-gated)
Reduce NonProd CR →
release quota to shared
pool

Full Engine State Machine (normative)

`engine_mode` is not a two-value flag but a **five-state machine** persisted in Cosmos DB with conditional writes, transition guards, and recovery tests — a production blocker until implemented: `[Documented]`

State	Meaning	Permitted transitions
STEADY_STATE	Organic growth only; Emergency Transfer rejected	→ DR_DECLARATION_PENDING
DR_DECLARATION_PENDING	DR requested, awaiting dual approval + validation	→ DR_EVENT_ACTIVE (approved) · → STEADY_STATE (rejected/expired)
DR_EVENT_ACTIVE	Crisis operations only; auto-increase suppressed	→ FAILBACK_PENDING · → INCIDENT_HOLD
FAILBACK_PENDING	Failback requested, being validated	→ STEADY_STATE (completed) · → DR_EVENT_ACTIVE (validation failed) · → INCIDENT_HOLD
INCIDENT_HOLD	State conflict / critical failure — safe hold	→ DR_EVENT_ACTIVE · → FAILBACK_PENDING (on recovery approval)

All transitions are **operator-gated with dual approval** — never automatic.



`EngineModeState` Entity

Must carry: environment/control-plane scope · current mode · state version · incident ID · requested-by · approved-by · transition timestamp · transition reason · active operation IDs · lease owner + expiry · recovery checkpoint. `[Documented]`

DR Activation Semantics

Entering `DR_EVENT_ACTIVE` **only establishes the operating mode** in which separately-governed emergency operations may be evaluated — it does **not** automatically authorize Tier 2 or Tier 3. Each tier is independently gated. The DR orchestrator validates group and subscription quota and CR/sharing state before starting any approved failover deployment, and records active-or-incident-hold state back to the state service. `[Documented]`

Consequences

Positive:

- Idle NonProd capacity doubles as DR overflow, cutting the cost of pre-positioned DR reserve while HC-6 guarantees sufficiency.
- The `engine_mode` gate makes it structurally impossible for routine reconciliation to trigger destructive VM operations.
- Tier 2 delivers meaningful crisis capacity **with zero VM disruption and zero Azure quota wait** — the common escalation path avoids Tier 3 entirely.
- Every operation is tagged with its operating mode for a clean audit trail.

Negative / trade-offs:

- Co-location adds risk that NonProd over-consumption erodes DR overflow; mitigated by HC-6, `dr_overflow_reserve`, and floor alerting.

- `engine_mode` must be a formal Cosmos DB state propagated to every reconciliation cycle, with operator-gated, dual-approval transitions.
- **Tier 3 is blocked** until the consumer-credential model (Managed Identity vs cross-tenant service-principal) and the engine-mode state machine are resolved — a known Phase-1 limitation.
- Tier 3 requires a rare, audited break-glass role (`ACRME.SuperAdmin`) for single-approver override.
- Requires per-CRG-type `RegionalSnapshot` fields to evaluate HC-6.

Alternatives Considered

Alternative	Why Rejected
Keep DR_region ≠ NonProd_region	Prevents using NonProd as DR overflow — the whole point of co-location. `[Decided]`
Remove separation with no compensating HC	Risks NonProd over-consumption leaving no DR headroom. `[Decided]`
Unified capacity engine with mode flags	Mode flags create complex conditionals and risk triggering VM disassociation during routine reconciliation. `[Decided]`
Emergency transfer as an extension of auto-increase	Conflates policy-driven growth with operator-gated crisis response; could run destructive ops without a DR declaration. `[Decided]`
Two tiers only (automated + destructive)	Loses the quota-neutral Tier 2 — the critical low-risk intermediate that avoids Tier 3 in most crises. `[Decided]`
Four tiers (separate VMSS tier)	VMSS disassociation deferred as a Phase-1 limitation rather than a distinct tier. `[Decided]`

ADR-004 — Forecast and Increase of Capacity and Quota

Status: Accepted
Date: August 2026
Deciders: Principal Cloud Architect, Platform Engineering, FinOps, Capacity Planning
Related constraints: HC-3 (applied at increase-execution time)

Context

Reserved capacity must grow ahead of demand — but Azure quota increases take time to approve, and over-reservation wastes money. The engine needs to forecast demand, recommend right-sized capacity, and drive quota/CR increases with enough lead time, **without** ever autonomously performing material or destructive changes in Phase 1. This growth path must be architecturally distinct from crisis operations (see ADR-003).

Decision

Adopt **forecast-driven, approval-gated capacity growth** operating exclusively in `STEADY_STATE`:

1. **Forecasting** analyses historical CR allocation and projects demand over a configurable window (default 30/60/90 days), exposing raw time series and derived recommendations via API. `[Documented]`

2. **Capacity sizing formula:** `[Documented]`
   `Forecast_Quantity = ceil(Forecast_Peak × (1 + Growth_Buffer) + DR_Buffer)`
   Increase when forecast demand exceeds current reserved quantity within the window; right-size down when demand is consistently below current, honouring a configurable buffer.

3. **Lead-time alerting:** when forecast demand approaches a quota limit (default **80%**), emit `ForecastApproachingQuotaLimit` with a **14-day lead time** — enough for quota-increase processing. `[Documented]`

4. **Auto-increase is approval-gated in Phase 1.** The trigger uses utilisation thresholds with **debounce/cooldown** to avoid thrashing; a `CapacityIncreaseRequest` entity carries the full lifecycle (create → approve → execute → retry → cancel), and Phase 1 requires **operator approval** before execution. `[Decided]`

5. **Quota increases** are initiated via the Azure Support REST API (`Microsoft.Capacity/.../serviceLimits`) — at group level where Quota Groups apply (see ADR-002) — subject to the same operator-approval gate. `[Documented]`

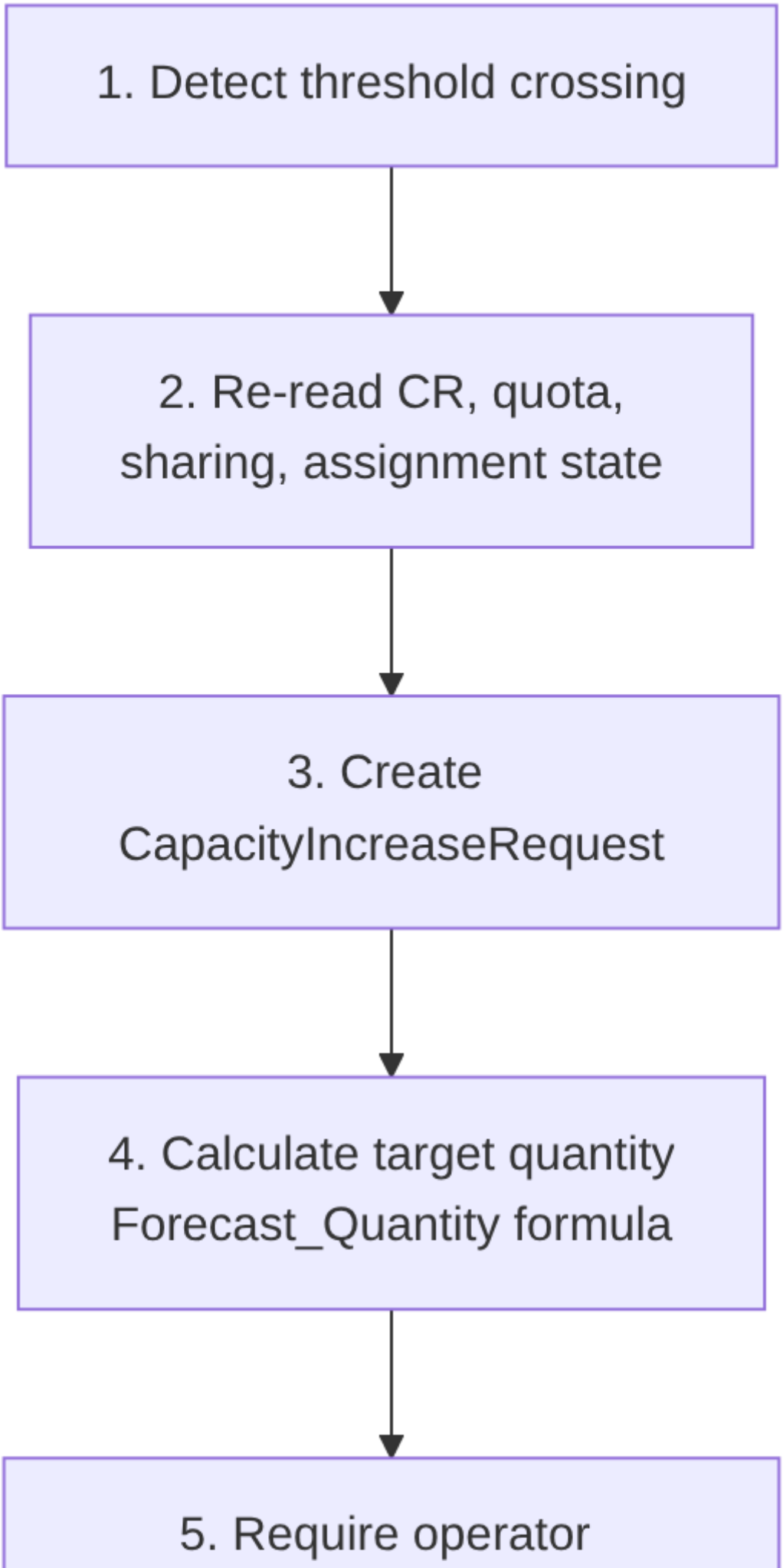
6. **Mode isolation.** Auto-increase runs **only** in `STEADY_STATE` and is **suppressed during** **`DR_EVENT_ACTIVE`**, keeping organic growth strictly separate from crisis transfer (see ADR-003). `[Decided]`

7. **Non-destructive guarantee.** Increases only ever raise CR quantity or request more quota; guarded reduction (right-sizing) never drops a CR below its allocated count (the platform floor) and is itself approval-gated.

Steady-State Capacity Lifecycle (normative 10-step policy)

The steady-state increase runs strictly separate from DR crisis operations and follows a fixed sequence, with **no assumed quota-propagation SLA**: `[Documented]`

1. Detect threshold crossing.
2. Re-read current CR, quota, sharing, and assignment state.
3. Create `CapacityIncreaseRequest`.
4. Calculate target quantity (`Forecast_Quantity` formula above).
5. **Require operator approval (Phase 1).**
6. Submit the quota action only if validated as required.
7. Wait for confirmed quota state **without assuming a propagation SLA**.
8. Update CR quantity.
9. Confirm the actual quantity.
10. Refresh the snapshot and close the request.


1. Detect threshold crossing
2. Re-read CR, quota, sharing, assignment state
3. Create CapacityIncreaseRequest
4. Calculate target quantity Forecast_Quantity formula
5. Require operator

Auto-Decrease Exclusion

Auto-decrease is **excluded from Phase 1**: it can remove future capacity and interact with running VMs. Right-sizing down remains operator-driven and guarded by the platform floor (never below allocated count). `[Documented]`

Forecast Advisory Posture

Forecast recommendations stay **advisory until model accuracy and false-positive rates are measured**; the horizon is 30/60/90 days and `Growth_Buffer` / `DR_Buffer` are policy percentages, not fixed constants. `[Documented]`

Consequences

Positive:
- Capacity grows ahead of demand with sufficient lead time for quota approvals — reducing capacity-exhaustion incidents.
- The debounce/cooldown and approval gate prevent runaway or thrashing increases.
- `CapacityIncreaseRequest` gives a fully auditable, retryable, cancellable growth workflow.
- Right-sizing recovers cost from over-reserved CRs without risking allocated VMs.

Negative / trade-offs:
- Approval-gated in Phase 1 means growth is not instantaneous — acceptable because Emergency Transfer (see ADR-003) covers crisis speed.
- Forecast accuracy depends on history; workload-tagged per-workload forecasts are only partially covered.
- Threshold, buffer, and cooldown values are empirical and require tuning during the pilot; SLA/propagation times are measured, not invented.
- the `CapacityIncreaseRequest` lifecycle (entity, approval, retry, cancellation) is still a backlog item requiring an end-to-end approved-increase test.

Alternatives Considered

Alternative	Why Rejected
Fully autonomous auto-increase	Removes human control over material cost/quota changes; unacceptable in Phase 1. `[Documented]`
Auto-increase escalating into emergency tiers	Conflates growth with crisis response; could run emergency ops without a DR declaration. `[Decided]`
Fixed-timer cooldown	Less adaptive than recovering the budget incrementally using observed success rates. `[Assumed]`
No lead-time alerting (react on exhaustion)	Quota approvals take too long; reacting at exhaustion guarantees deployment failures. `[Documented]`
Sizing without a DR buffer	Under-reserves relative to DR obligations; the formula includes an explicit `DR_Buffer` term. `[Documented]`

Appendix A — ADR Summary

ADR	Hard Constraints Applied	Key Open Items
ADR-001 Region Selection	HC-1, HC-4, HC-5, HC-8, HC-9, HC-10	Worked scoring examples pending final per-CRG-type inputs
ADR-002 Quota & Capacity	HC-2, HC-3, HC-7	Azure Quota Groups GA validated by proof-of-concept before rollout
ADR-003 Capacity during DR	HC-1, HC-4, HC-6	Consumer-credential model and engine-mode state machine; quota-release latency measured
ADR-004 Forecast & Increase	HC-3 (at execution)	Workload-tagged forecasts; `CapacityIncreaseRequest` lifecycle end-to-end test

Appendix B — Status Legend

Status	Meaning
Proposed	Under discussion; not yet ratified
Accepted	Ratified and in force
Deprecated	No longer recommended but not yet replaced
Superseded	Replaced by a later ADR (referenced explicitly)

Appendix C — Evidence Tag Taxonomy

Tag	Meaning
`[Documented]`	Traceable to Azure platform behaviour or documentation
`[Decided]`	An explicit ACRME design choice recorded in this ADR set
`[Derived]`	A logical consequence of a documented constraint or decision
`[Assumed]`	Architectural judgement pending proof-of-concept validation

---

Document Status: Accepted
Next Review: After proof-of-concept validation of Azure Quota Groups GA and quota-release latency, and on resolution of the consumer-credential model and engine-mode state-machine items.